Relational Foundations — Test Bank & Lab Challenges

Reference schema (run this first)

```
DROP TABLE IF EXISTS Enrollment, Course, Student, Department CASCADE;

CREATE TABLE Department (
    DeptID    INT          PRIMARY KEY,
    DeptName  VARCHAR(100) NOT NULL UNIQUE
);

CREATE TABLE Student (
    StudentID INT          PRIMARY KEY,
    Name      VARCHAR(100) NOT NULL,
    Email     VARCHAR(150) NOT NULL UNIQUE,
    DeptID    INT          REFERENCES Department(DeptID)
                           ON DELETE SET NULL,      -- student survives, loses department
    Profile   JSONB                                 -- semi-structured extension
);

CREATE TABLE Course (
    CourseID  INT          PRIMARY KEY,
    Title     VARCHAR(150) NOT NULL,
    Credits   SMALLINT     NOT NULL CHECK (Credits BETWEEN 1 AND 4),
    DeptID    INT          NOT NULL,
    CONSTRAINT fk_course_dept FOREIGN KEY (DeptID)
        REFERENCES Department(DeptID) ON DELETE RESTRICT  -- a dept with courses cannot vanish
);

CREATE TABLE Enrollment (
    StudentID INT         REFERENCES Student(StudentID) ON DELETE CASCADE,
    CourseID  INT         REFERENCES Course(CourseID)   ON DELETE RESTRICT,
    Semester  VARCHAR(20) NOT NULL,
    Grade     CHAR(2)     CHECK (Grade IN ('A','A-','B+','B','B-','C+','C','D','F')),
    PRIMARY KEY (StudentID, CourseID, Semester)
);
```

Three design decisions:

- `Grade` allows NULL, because an in-progress enrollment has no grade yet. NULL means *unknown*, not *zero*.
- Unquoted identifiers fold to lower case, so PostgreSQL reports `course` and `course_credits_check`, not `Course`.
- The column-level `CHECK` gets an auto-generated name; `fk_course_dept` is named explicitly so error messages are readable. Name every constraint in production DDL.

Part 2 · Exercise 1: Negative testing (break it on purpose)

Run each statement, read the error, and name the constraint and its SQLSTATE code, which is what application code actually catches.

```
-- Setup
INSERT INTO Department (DeptID, DeptName) VALUES (1, 'Computer Science');
INSERT INTO Student (StudentID, Name, Email, DeptID)
VALUES (1000, 'Rafiq Islam', 'rafiq@univ.edu', 1);
```

```
-- T1 Primary key
INSERT INTO Department (DeptID, DeptName) VALUES (1, 'Mathematics');

-- T2 CHECK (domain)
INSERT INTO Course (CourseID, Title, Credits, DeptID)
VALUES (101, 'Advanced Database Systems', 6, 1);

-- T3 Foreign key
INSERT INTO Student (StudentID, Name, Email, DeptID)
VALUES (5001, 'John Doe', 'john@univ.edu', 999);

-- T4 UNIQUE
INSERT INTO Student (StudentID, Name, Email, DeptID)
VALUES (5002, 'Rafiq Hasan', 'rafiq@univ.edu', 1);

-- T5 NOT NULL
INSERT INTO Course (CourseID, Title, Credits, DeptID)
VALUES (102, NULL, 3, 1);
```

Test	Expected PostgreSQL error	Constraint	SQLSTATE
T1	duplicate key value violates unique constraint "department_pkey" — Key (deptid)=(1) already exists	Primary key	23505
T2	new row for relation "course" violates check constraint "course_credits_check"	CHECK	23514
T3	insert or update on table "student" violates foreign key constraint "student_deptid_fkey" — Key (deptid)=(999) is not present in table "department"	Foreign key	23503
T4	duplicate key value violates unique constraint "student_email_key"	UNIQUE	23505
T5	null value in column "title" of relation "course" violates not-null constraint	NOT NULL	23502

Trap tests: these succeed, and students must explain why

```
-- Trap A: three-valued logic in CHECK
INSERT INTO Enrollment VALUES (1000, 101, 'Fall 2026', NULL);   -- after a valid course 101 exists

-- Trap B: case sensitivity defeats UNIQUE
INSERT INTO Student (StudentID, Name, Email, DeptID)
VALUES (5003, 'Rafiq Islam', 'RAFIQ@univ.edu', 1);
```

- **Trap A succeeds.** `NULL IN ('A', ...)` evaluates to UNKNOWN, and a `CHECK` rejects only FALSE. This is SQL's three-valued logic: a constraint that must forbid NULL needs an explicit `NOT NULL`.
- **Trap B succeeds.** `'RAFIQ@univ.edu'` and `'rafiq@univ.edu'` are different strings, so the same mailbox now has two student records. The fix is in Part 3, challenge C1.

Go deeper. UNIQUE also lets multiple NULLs through, because NULL is never equal to NULL. Since PostgreSQL 15, `UNIQUE NULLS NOT DISTINCT` changes that, an option the SQL:2023 standard also adopted.

Part 2 · Exercise 2: JSONB and the converged relational model

`Profile JSONB` shows that a modern relational table can hold semi-structured data without giving up SQL, transactions or indexes.

Base tasks

```
-- E2.1 Insert semi-structured data
INSERT INTO Student (StudentID, Name, Email, DeptID, Profile)
VALUES (1001, 'Alice Rahaman', 'alice@univ.edu', 1,
  '{"emergency_contact": {"name": "Farhana Rahaman", "relation": "Mother",
                          "phone": "+8801700000000"},
    "interests": ["AI", "Cybersecurity"]}'::jsonb);

-- E2.2 Navigate nested fields
SELECT Name,
       Profile -> 'emergency_contact' ->> 'phone' AS emergency_phone
FROM Student
WHERE StudentID = 1001;

-- E2.3 Containment search
SELECT Name, Email
FROM Student
WHERE Profile @> '{"interests": ["AI"]}';
```

Model answers.

E2.2 returns `Alice Rahaman | +8801700000000`. The key point is the operator pair: `->` returns `jsonb` (keep navigating), `->>` returns `text` (finish and compare).

Ask: what does `Profile -> 'emergency_contact' -> 'phone'` return? Answer: `"+8801700000000"` *with* quotes, a jsonb string, which will not equal a text literal in a `WHERE`.

E2.3 matches because `@>` tests whether the left document *contains* the right one; array containment ignores order and extra elements.

Stretch tasks

E2.4 · Make containment fast. Load 100,000 synthetic students, run `EXPLAIN ANALYZE` on E2.3, add an index, and run it again.

```
INSERT INTO Student (StudentID, Name, Email, DeptID, Profile)
SELECT g, 'Student ' || g, 's' || g || '@univ.edu', 1,
       jsonb_build_object('interests',
         CASE WHEN g % 50 = 0 THEN '["AI"]'::jsonb ELSE '["Networks"]'::jsonb END)
FROM generate_series(10000, 109999) AS g;

CREATE INDEX idx_student_profile ON Student USING GIN (Profile jsonb_path_ops);
ANALYZE Student;
```

Expected: the plan changes from `Seq Scan` to `Bitmap Index Scan on idx_student_profile`. A GIN index is an *inverted index*, the same structure behind search engines: it maps each key-value path to the rows that contain it. `jsonb_path_ops` is smaller and faster for `@>`; the default `jsonb_ops` also supports the key-existence operators `?`, `?|`, `?&`.

E2.5 · Standard SQL/JSON. Rewrite E2.2 without PostgreSQL-specific operators, then shred the interests array into rows.

```
SELECT Name,
       JSON_VALUE(Profile, '$.emergency_contact.phone') AS emergency_phone
FROM Student WHERE StudentID = 1001;

SELECT s.Name, jt.interest
FROM Student s,
     JSON_TABLE(s.Profile, '$.interests[*]'
                COLUMNS (interest TEXT PATH '$')) AS jt
WHERE s.StudentID = 1001;
```

`JSON_TABLE` (PostgreSQL 17+) turns a JSON array back into a relation, so the result is two rows, `AI` and `Cybersecurity`, and can be joined like any table. This is the bridge between document and relational thinking.

E2.6 · Put a schema back on schema-on-read. Stop a client from storing `interests` as a string instead of an array.

```
ALTER TABLE Student ADD CONSTRAINT chk_profile_interests
  CHECK (Profile IS NULL
         OR NOT (Profile ? 'interests')
         OR jsonb_typeof(Profile -> 'interests') = 'array');

-- Must now fail with 23514:
UPDATE Student SET Profile = '{"interests": "AI"}' WHERE StudentID = 1001;
```

Discussion question. When should a JSON field be promoted to a real column?

Expected answer: when it is filtered or joined on constantly, needs a foreign key or `UNIQUE`, or is sensitive. The emergency phone number here is personal data hidden inside a blob, which is harder to mask, audit or encrypt than a dedicated column. Flexibility is borrowed against future integrity.

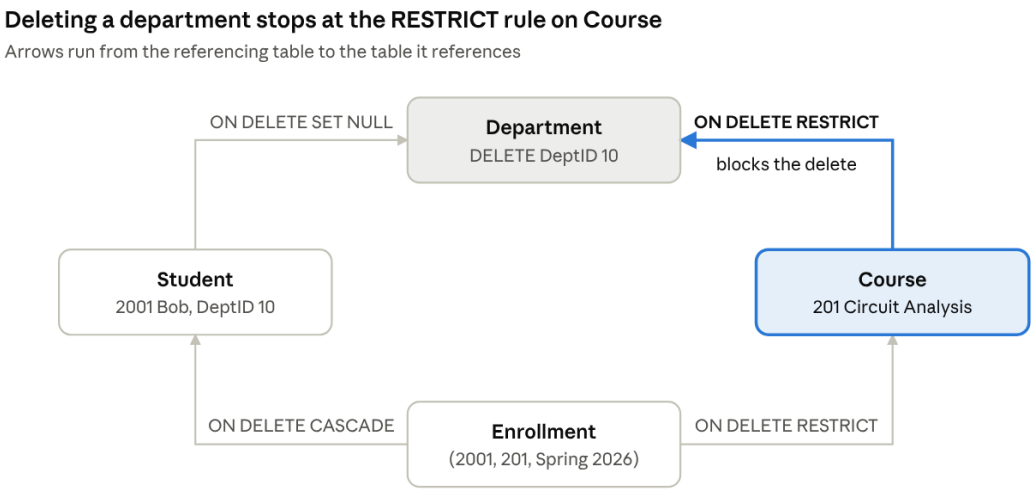
Part 2 · Exercise 3: Cascading execution challenge

The `DELETE` fails: `Course 201` still references department 10 through `fk_course_dept`, which is `RESTRICT`. Students must predict this *before* running it.

```
-- Step A: seed one department, course, student and enrollment
INSERT INTO Department (DeptID, DeptName) VALUES (10, 'Electrical Engineering');
INSERT INTO Course (CourseID, Title, Credits, DeptID) VALUES (201, 'Circuit Analysis', 3, 10);
INSERT INTO Student (StudentID, Name, Email, DeptID) VALUES (2001, 'Bob Smith', 'bob@univ.edu',
10);
INSERT INTO Enrollment (StudentID, CourseID, Semester, Grade) VALUES (2001, 201, 'Spring 2026',
'A');

-- Step B: attempt to delete the department
DELETE FROM Department WHERE DeptID = 10;
```

Expected error (SQLSTATE 23503): update or delete on table "department" violates foreign key constraint "fk_course_dept" on table "course" — Key (deptid)=(10) is still referenced from table "course".



Foreign-key graph of the reference schema · 4 tables, 4 referential actions

PostgreSQL walks every foreign key that points at the deleted row; the first rule that refuses, Course's `RESTRICT`, aborts the whole statement.

Answer key

1. **Does it succeed?** No. A department that still offers courses cannot be deleted.
2. **Which table blocked it?** `Course`, through `fk_course_dept ON DELETE RESTRICT`. `Student` would have cascaded `SET NULL`, but it never matters.
3. **The question that separates strong students:** was Bob's `DeptID` set to NULL before the error? Prove it:

```
SELECT StudentID, DeptID FROM Student WHERE StudentID = 2001;   -- still 10
```

It is still 10. A SQL statement is atomic: when one referential action fails, every action the same statement triggered is rolled back. This is the A in ACID, met here in Week 3 before the transactions session names it.

Follow-up runs

Variant	Change	Outcome
F1	Delete in dependency order: Enrollment, then Course 201, then Department 10	Succeeds; Bob's `DeptID` becomes NULL through `SET NULL`
F2	Redefine `fk_course_dept` as `ON DELETE CASCADE`, repeat Step B	Still fails, now on `enrollment_courseid_fkey`: the cascade deletes Course 201, then hits Enrollment's `RESTRICT`
F3	As F2, but also make `Enrollment.CourseID` `ON DELETE CASCADE`	Succeeds and silently destroys Bob's grade, which is why academic records should never sit at the end of a cascade chain

F3 is the lesson: `CASCADE` is convenient in a lab and dangerous on records with legal or audit value. A safe architecture combines `RESTRICT` on history with soft deletes on master data.

Part 3 · Advanced extension challenges

Five stretch challenges show where constraint design stands in 2026: integrity rules now cover case, time, transactions, legacy data and distributed keys. Most need PostgreSQL 18 (released September 2025); each lists the classic idea it extends.

#	Challenge	Builds on	Needs
C1	Case-insensitive email uniqueness	UNIQUE, Trap B	Any version
C2	No overlapping department heads	Composite keys	PostgreSQL 18
C3	Circular foreign keys	Referential integrity, atomicity	Any version
C4	Constraints on dirty legacy data	CHECK, FK	NOT VALID: any; NOT ENFORCED: 18
C5	Time-ordered UUID keys	Surrogate keys (Q3)	PostgreSQL 18

C1 · Make `Email` unique regardless of case

Fix Trap B from Exercise 1.

```
DELETE FROM Student WHERE StudentID = 5003;          -- remove the duplicate first
CREATE UNIQUE INDEX uq_student_email_ci ON Student (lower(Email));
```

An *expression index* enforces uniqueness on a computed value, not the stored one. Alternatives worth naming: the `citext` extension type, or a nondeterministic ICU collation. The unique index fails to build while duplicates exist, so cleaning data precedes adding constraints.

C2 · Temporal primary key: one head per department at any time

Before PostgreSQL 18 this needed an `EXCLUDE USING gist` constraint. Now the key itself understands time.

```
CREATE EXTENSION IF NOT EXISTS btree_gist;

CREATE TABLE DeptHead (
    DeptID      INT       REFERENCES Department(DeptID),
    HeadName    VARCHAR(100) NOT NULL,
    ValidPeriod DATERANGE NOT NULL,
    PRIMARY KEY (DeptID, ValidPeriod WITHOUT OVERLAPS)
);

INSERT INTO DeptHead VALUES (1, 'Dr. Karim', '[2024-01-01, 2026-07-01)');
INSERT INTO DeptHead VALUES (1, 'Dr. Sultana', '[2026-07-01, 2028-07-01)');  -- succeeds: periods
touch, no overlap
INSERT INTO DeptHead VALUES (1, 'Dr. Ahmed', '[2026-01-01, 2027-01-01)');    -- fails: overlaps
both
```

The key is no longer "no two equal rows" but "no two rows whose periods overlap". Why the range is written `[start, end)`: half-open intervals let consecutive terms meet without overlapping. PostgreSQL 18 also adds temporal foreign keys with `PERIOD`, so a child row's validity must sit inside its parent's.

C3 · Circular foreign keys and deferred checking

Every department has a chair, and every instructor belongs to a department. Neither row can be inserted first.

```
CREATE TABLE Instructor (
    InstructorID INT PRIMARY KEY,
    Name         VARCHAR(100) NOT NULL,
    DeptID       INT NOT NULL REFERENCES Department(DeptID)
                 DEFERRABLE INITIALLY DEFERRED
);
ALTER TABLE Department ADD COLUMN ChairID INT
    REFERENCES Instructor(InstructorID) DEFERRABLE INITIALLY DEFERRED;

BEGIN;
INSERT INTO Department (DeptID, DeptName, ChairID) VALUES (20, 'Data Science', 900);
INSERT INTO Instructor VALUES (900, 'Dr. Nusrat', 20);
COMMIT;   -- both foreign keys are checked here, and pass
```

Inside the transaction the database is briefly inconsistent; at `COMMIT` it must be consistent again. This is the C in ACID made concrete, and it is why `RESTRICT` (never deferrable) and `NO ACTION` differ.

C4 · Adding rules to dirty legacy data

A migrated table already holds rows with `Credits = 0`. Students compare two strategies.

```
-- Strategy 1: enforce for new rows now, clean and validate later
ALTER TABLE Course ADD CONSTRAINT chk_credits_v2
    CHECK (Credits BETWEEN 1 AND 5) NOT VALID;
-- ... fix bad rows ...
ALTER TABLE Course VALIDATE CONSTRAINT chk_credits_v2;

-- Strategy 2 (PostgreSQL 18): declare the rule, enforce nothing
ALTER TABLE Course ADD CONSTRAINT chk_credits_doc
    CHECK (Credits BETWEEN 1 AND 5) NOT ENFORCED;
```

`NOT VALID` avoids a long table scan and lock on a large production table. `NOT ENFORCED` records the business rule for documentation tools, data catalogs and AI query generators without rejecting anything, the same idea as the informational constraints in Snowflake and BigQuery. Which strategy would you trust for grades?

Part 4 · Practical session defense checklist

#	Prompt	Expected evidence
D1	Show the line that stops a student enrolling in the same course twice in one semester.	`PRIMARY KEY (StudentID, CourseID, Semester)` on `Enrollment`; then they run a duplicate insert and show error 23505
D2	Show how your schema handles students if their department is dissolved.	`ON DELETE SET NULL` on `Student.DeptID`; they justify why a student must survive the department
D3	Why can your `Course` rows never end up with a NULL department?	`Course.DeptID NOT NULL` plus `RESTRICT`; they explain why `SET NULL` would contradict `NOT NULL`

#	Prompt	Expected evidence
D4	Prove a 6-credit course cannot be stored, even through psql.	They run the insert live and show `course_credits_check`
D5	Show me every constraint on `Student` without opening your DDL file.	`\d student`, or a query on `information_schema.table_constraints`
D6	Insert a grade of 'A+'. What happens, and is that the right rule for this university?	23514 from the `Grade` CHECK; a strong group discusses whether grades belong in a lookup table instead
D7	Your `Profile` holds an emergency phone number. Defend keeping it in JSONB, or move it.	Either answer passes if argued from querying, constraints and privacy
D8	Delete department 10 with enrollments present and tell me what changed.	Nothing changed: statement atomicity, as in Exercise 3